19.01 Logic circuits

Chapter 4 introduced the symbols for logic gates that are used in logic circuits and discussed the relationships between logic circuits, truth tables and logic expressions. This chapter introduces some specific circuits that are used to construct components that provide functionality in computer hardware.

The half adder

A fundamental operation in computing is binary addition. The result of adding two bits is either 1 or 0. However, when 1 is added to 1 the result is 0 but there is a carry bit equal to 1. This cannot be ignored if two numbers with several bits in each are being added.

The simplest circuit that can be used for binary addition is the **half adder**. This can be represented by the diagram in Figure 19.01. The circuit takes two input bits and outputs a sum bit (S) and a carry bit (C).

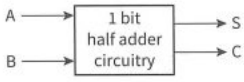


Figure 19.01 A half adder

The truth table for this circuit is shown in Table 19.01.

Input		Output	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Table 19.01 The truth table for a half adder

One possible circuit can be defined directly by examination of the truth table. It can be seen that the only combination of inputs that produces a 1 for the carry bit is when two 1 bits are input. The truth table for the C output is in fact the AND truth table. The truth table for the S output can be seen to match that for the XOR operator which is shown in Figure 4.02 in Chapter 4 (Section 4.04). Therefore, one circuit that would produce the half adder functionality would contain an AND gate and an XOR gate with each gate receiving input from A and B.

This is only one of several circuits that would provide the functionality. The NAND and NOR gates are universal gates. Any logic circuit can be constructed using only NAND gates or only NOR gates. This fact combined with the ease of manufacture of these gates leads circuit manufacturers to prefer their use. The circuit shown in Figure 19.02 consisting only of NAND gates has the correct logic to produce the C and S outputs and is a likely choice for implementation.

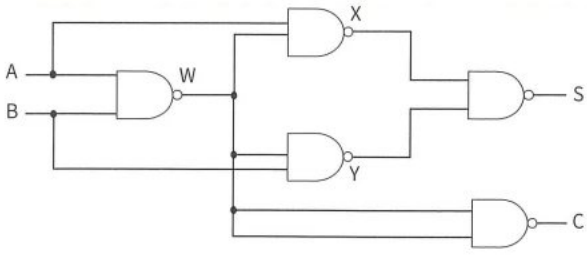


Figure 19.02 A half adder circuit constructed from NAND gates

Question 19.01

In Figure 19.02, can you identify the individual circuits that represent the AND operator and the XOR operator?

TASK 19.01

Use the intermediate points labelled W, X and Y to construct a truth table for the circuit shown in Figure 19.02. Check that this reproduces the truth table shown as Table 19.01.

The full adder

If two numbers expressed in binary with several bits to be added, the addition must start with the two least significant bits and then proceed to the most significant bits. At each stage, the carry from the previous addition has to be incorporated into the current addition. If a half adder is used each time, there has to be separate circuitry to handle the carry bit because the half adder only takes two inputs.

The **full adder** is a circuit that has three inputs including the previous carry bit. The truth table is shown as Table 19.02.

Input			Output	
A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 19.02 The truth table for a full adder

One possible circuit for implementation contains two half adder circuits and an OR gate as shown in Figure 19.03.

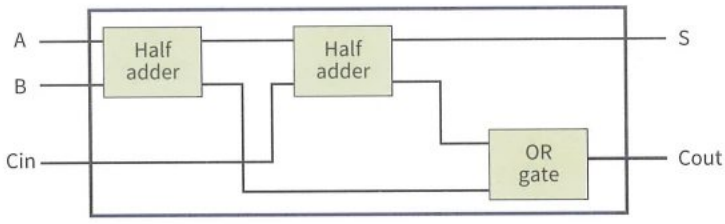


Figure 19.03 A possible implementation of a full adder

KEY TERMS

Half adder circuit: a circuit which performs binary addition of two individual bits

Full adder circuit: a circuit which performs binary addition of two individual bits and an input carry bit

As before, it is possible to construct the circuit entirely from NAND gates as shown in Figure 19.04.

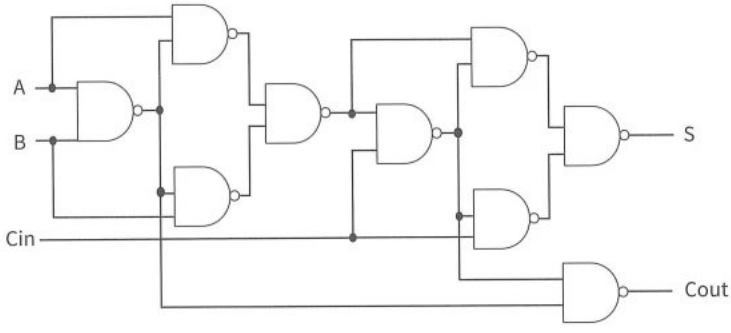


Figure 19.04 A full adder circuit constructed from NAND gates

Discussion Point:

Can you see how full adders could be combined to handle addition of, for example, four-bit binary numbers? What happens to the carry input for the first addition?

19.02 Sequential logic circuits

All of the circuits so far encountered in this book have been **combinational circuits**. For such a circuit the output is dependent only on the input values. An alternative type of circuit is a **sequential circuit** where the output depends on the input and on the previous output.

KEY TERMS

Combinational circuit: a circuit in which the output is dependent only on the input values

Sequential circuit: a circuit in which the output depends on the input values and the previous output

The SR flip-flop

The SR flip-flop or 'latch' is a simple example of a sequential circuit. It can be constructed with two NAND gates or two NOR gates. Figure 19.05 shows the version with two NOR gates

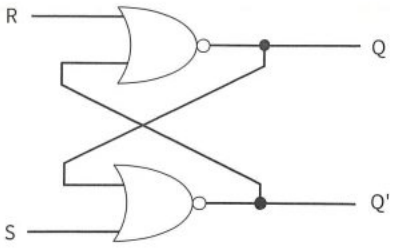


Figure 19.05 A circuit for an SR flip-flop using NOR gates

The cross-coupled circuit with output feedback has some interesting consequences. For example, if we consider the hypothetical situation where R, S, Q and Q' all had value 0 we can see that this would be self-contradictory because two inputs value 0 to a NOR gate produce a 1 output. A state with Q and Q' both set to value 1 would be self-contradictory because the output from a NOR gate is always 0 if either input is 1. The flip-flop is therefore a two-state device. Either it has Q=1 and Q'=0 or it has the reverse.

The truth table for the circuit can be presented as shown in Table 19.03. The two lines of the truth table where both S and R are input as 0 represent the condition when no signal is input to the flip-flop. If we consider the state with Q=1 and Q'=0:

- and we consider the condition that both S and R inputs are 0
- then the top NOR gate has inputs both 0
- giving output 1
- and the bottom NOR gate has inputs 1 and 0
- giving output 0.

This means that this is a self-consistent state which is referred to as the set state. A similar argument can be applied for the alternative to the set state which is the unset state with Q=0 and Q'=1.

The S and R inputs are for set and reset, respectively. The truth table shows that an input combination of S=0 and R=1 converts a set state to an unset state and an input combination of S=1 and R=0 converts an unset state to a set state.

Input signals		Initial state		Final state	
S	R	Q	Q'	Q	Q'
0	0	1	0	1	0
1	0	1	0	1	0
0	1	1	0	0	1
0	0	0	1	0	1
1	0	0	1	1	0
0	1	0	1	0	1

Table 19.03 A representation of a truth table for an SR flip-flop

These properties explain why the SR flip-flop can be used as a storage device for 1 bit and therefore could be used as a component in RAM because a value is stored but can be altered. The truth table does not contain rows for R=1 and S=1 because this leads to an invalid

state with both Q and Q' having value 0. Because of this the circuit must be protected from receiving an input signal on R and S simultaneously.

The alternative NAND gate circuit for the SR flip-flop has a similar structure but the labeling is different. The important difference is that setting is achieved with S=0 and R=1 and resetting with R=0 and S=1.

TASK 19.02

Find a diagram for a NAND gate SR flip-flop and construct the truth table.

The JK flip-flop

In addition to the possibility of entering an invalid state there is also the potential for a circuit to arrive in an uncertain state if inputs do not arrive quite at the same time. In order to prevent this, a circuit may include a clock pulse input to give a better chance of synchronising inputs. The JK flip-flop is an example.

The JK flip-flop can be illustrated by the symbol shown in Figure 19.06(a). A possible circuit is shown in Figure 19.06(b).

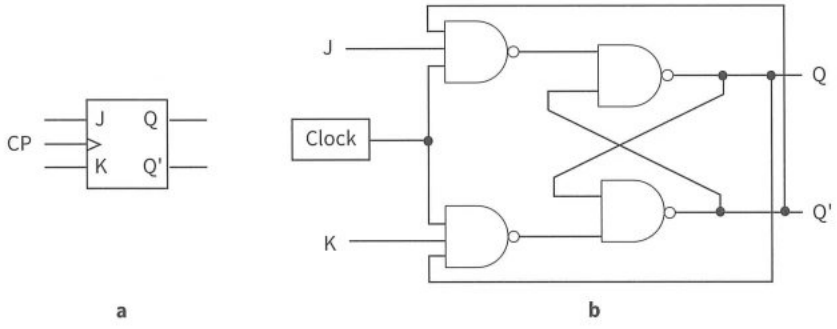


Figure 19.06 (a) A symbol for a JK flip-flop and (b) a possible circuit

TIP

There is one special case for a NAND gate; when all inputs are 1 the output is 0. There is one special case for a NOR gate; when all inputs are 0 the output is 1.

To understand the workings of the circuit you must first remember that a NAND gate gives a 1 output unless all inputs are 1. If the circuit is in the unset state with Q = 0 and Q' =1 this is stable and self-consistent as shown by the following argument:

- the clock and R both input 0
- the top-left NAND gate therefore has output 1
- this then leads to two 1 inputs to the top-right NAND gate
- ensuring that Q = 0.

If the input from J now becomes 1 and the clock pulse switches to a 1 then:

- the top-left NAND gate has all 1 inputs
- so the output is 0
- this causes the top-right NAND gate to output a 1 for Q

- there are now two 1 inputs to the bottom-right NAND gate
- so the Q' output becomes 0.

This shows how the J input is a set input. A similar argument shows that the K is a clear input. In this respect the JK flip-flop behaves in a similar way to the SR flip-flop as a storage device for one bit. However, there is an important difference in that if both J and K are input as a 1 then the values for Q and Q' are toggled (they switch value). This makes the JK flip-flop a more reliable device because there is no combination of input states that leave uncertainty as to which values are stored. The significant part of the truth table is shown as Table 19.04.

J	K	Clock	Q
0	0	↑	Q unchanged
1	0	↑	1
0	1	↑	0
1	1	↑	Q toggles

Table 19.04 Part of the truth table for a JK flip-flop

Question 19.02

Can you follow the logic in the JK flip-flop circuit to see that the truth table is an accurate representation of the logic for all combinations of input?

19.03 Boolean algebra basics

Chapter 4 (Section 4.01) introduced logic expressions consisting of logic propositions combined using Boolean operators. Boolean algebra provides a simplified way of writing a logic expression and a set of rules for manipulating an expression.

Whenever a form of algebra is used it is vital that there is an understanding of its meaning. As a simple example we can consider the following four interpretations of the meaning of 1 + 1:

```
1 + 1 = 2
1 + 1 = 10
1 + 1 = 0
1 + 1 = 1
```

The first shows denary arithmetic, the second binary arithmetic and the third bit arithmetic. The last one applies if Boolean algebra is being used. This is because in Boolean algebra 1 represents TRUE, 0 represents FALSE, and + represents OR. Therefore the fourth statement represents the logic statement:

TRUE OR TRUE is TRUE

There are options for the representation of Boolean algebra. For example, the symbols for AND and OR are sometimes represented as ∧ and ∨. There is also the option of writing A.B or AB for A AND B. The dot notation is used in this book. Finally, there are options for how NOT A (the inverse of A) can be represented. $\bar{A}$ is used here.

TIP

You do not need to remember the ∧ and ∨ notation.